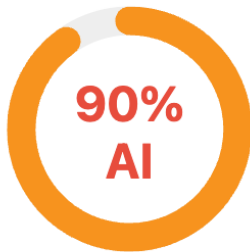


Detection Result



Likelihood of AI Generation: 90%

The code reveals typical AI generation features including uniform naming, lack of comments, textbook structuring of LSTM components, and generic data and training handling, making it highly probable that this code was generated by AI.

Naming Style: Function and variable names are highly standardized and generic, following typical AI naming conventions without business-specific or domain-specific terms.

Comment Style: The code lacks comments entirely, which is common in AI-generated code that focuses on functionality over contextual explanations.

Code Structure: The code is very neatly organized, stylistically uniform, with consistent indentation, spacing, and conventional code patterns found in AI-generated examples.

Typical AI Traits: The code uses helper functions like `sigmoid_deriv` and `tanh_deriv` unnecessarily; the naming and structure are textbook-like covering standard LSTM cells and linear layers commonly generated by AI.

Business Footprints: There is minimal real business logic; the data handling and training loop are generic and do not include complex error handling or domain-specific adaptations.

```
import numpy as np
```

```
def sigmoid(x):
```

```
    x = np.clip(x, -500, 500)
```

```
    return 1.0 / (1.0 + np.exp(-x))
```

```
def sigmoid_deriv(x):
```

```
    s = sigmoid(x)
```

```
    return s * (1.0 - s)
```

```
def tanh_deriv(x):
```

```
    t = np.tanh(x)
```

```
    return 1.0 - t ** 2
```

```
class LSTMCell:
```

```
def __init__(self, input_size, hidden_size, seed=None):

    rng = np.random.default_rng(seed)

    self.H = hidden_size

    self.I = input_size

    fan = self.H + self.I

    lim = 0.5 * np.sqrt(6.0 / (fan + 4 * self.H))

    self.W = rng.uniform(-lim, lim, size=(4 * self.H, fan)).astype(np.float64)

    self.b = np.zeros((4 * self.H, 1), dtype=np.float64)

    self.dW = np.zeros_like(self.W)

    self.db = np.zeros_like(self.b)

    self._adam_w = [np.zeros_like(self.W), np.zeros_like(self.W), 0]

    self._adam_b = [np.zeros_like(self.b), np.zeros_like(self.b), 0]


def zero_grad(self):

    self.dW.fill(0.0)

    self.db.fill(0.0)


def forward(self, x, h_prev, c_prev):

    z = np.vstack([h_prev, x])
```

```
a = np.dot(self.W, z) + self.b
```

```
H = self.H
```

```
ai, af, ag, ao = a[0:H], a[H : 2 * H], a[2 * H : 3 * H], a[3 * H : 4 * H]
```

```
i_gate = sigmoid(ai)
```

```
f_gate = sigmoid(af)
```

```
c_tilde = np.tanh(ag)
```

```
o_gate = sigmoid(ao)
```

```
c = f_gate * c_prev + i_gate * c_tilde
```

```
tanh_c = np.tanh(c)
```

```
h = o_gate * tanh_c
```

```
cache = {
```

```
    "z": z,
```

```
    "i_gate": i_gate,
```

```
    "f_gate": f_gate,
```

```
    "c_tilde": c_tilde,
```

```
    "o_gate": o_gate,
```

```
    "c": c,
```

```
    "tanh_c": tanh_c,
```

```
    "c_prev": c_prev,
```

```
    "ai": ai,  
  
    "af": af,  
  
    "ag": ag,  
  
    "ao": ao,  
  
}  
  
return h, c, cache
```

```
def backward(self, cache, dh_up, dc_up):
```

```
    H = self.H  
  
    i_gate = cache["i_gate"]  
  
    f_gate = cache["f_gate"]  
  
    c_tilde = cache["c_tilde"]  
  
    o_gate = cache["o_gate"]  
  
    c = cache["c"]  
  
    tanh_c = cache["tanh_c"]  
  
    c_prev = cache["c_prev"]  
  
    z = cache["z"]  
  
    ai, af, ag, ao = cache["ai"], cache["af"], cache["ag"], cache["ao"]
```

```
do = dh_up * tanh_c

d_tanh_c = dh_up * o_gate

dc = dc_up + d_tanh_c * tanh_deriv(c)

df = dc * c_prev

di = dc * c_tilde

dc_tilde = dc * i_gate

dc_prev = dc * f_gate


da = np.vstack(

    [

        di * sigmoid_deriv(ai),

        df * sigmoid_deriv(af),

        dc_tilde * tanh_deriv(ag),

        do * sigmoid_deriv(ao),

    ]

)

self.dW += np.dot(da, z.T)

self.db += da

dz = np.dot(self.W.T, da)
```

```
return dz[0:H], dc_prev
```

```
def update(self, lr):
```

```
    for P, G, slot in (
```

```
        (self.W, self.dW, self._adam_w),
```

```
        (self.b, self.db, self._adam_b),
```

```
    ):
```

```
        g = np.clip(G, -5.0, 5.0)
```

```
        slot[2] += 1
```

```
        t = slot[2]
```

```
        slot[0][:] = 0.9 * slot[0] + 0.1 * g
```

```
        slot[1][:] = 0.999 * slot[1] + 0.001 * (g ** 2)
```

```
        m_hat = slot[0] / (1.0 - 0.9**t)
```

```
        v_hat = slot[1] / (1.0 - 0.999**t)
```

```
        P -= lr * m_hat / (np.sqrt(v_hat) + 1e-8)
```

```
class LinearLayer:
```

```
    def __init__(self, in_features, out_features, seed=None):
```

```
rng = np.random.default_rng(seed)

lim = 0.5 * np.sqrt(6.0 / (in_features + out_features))

self.W = rng.uniform(-lim, lim, size=(out_features, in_features)).astype(np.float64)

self.b = np.zeros((out_features, 1), dtype=np.float64)

self.dW = np.zeros_like(self.W)

self.db = np.zeros_like(self.b)

self._mw = np.zeros_like(self.W)

self._vw = np.zeros_like(self.W)

self._tw = 0

self._mb = np.zeros_like(self.b)

self._vb = np.zeros_like(self.b)

self._tb = 0
```

```
def zero_grad(self):
```

```
    self.dW.fill(0.0)
```

```
    self.db.fill(0.0)
```

```
def forward(self, x):
```

```
    y = np.dot(self.W, x) + self.b
```

```
return y, {"x": x}
```

```
def backward(self, cache, dy):
```

```
    x = cache["x"]
```

```
    self.dW += np.dot(dy, x.T)
```

```
    self.db += dy
```

```
    return np.dot(self.W.T, dy)
```

```
def update(self, lr):
```

```
    g = np.clip(self.dW, -5.0, 5.0)
```

```
    self._tw += 1
```

```
    self._mw[:] = 0.9 * self._mw + 0.1 * g
```

```
    self._vw[:] = 0.999 * self._vw + 0.001 * (g ** 2)
```

```
    m_hat = self._mw / (1.0 - 0.9**self._tw)
```

```
    v_hat = self._vw / (1.0 - 0.999**self._tw)
```

```
    self.W -= lr * m_hat / (np.sqrt(v_hat) + 1e-8)
```

```
    g = np.clip(self.db, -5.0, 5.0)
```

```
    self._tb += 1
```

```
self._mb[:] = 0.9 * self._mb + 0.1 * g

self._vb[:] = 0.999 * self._vb + 0.001 * (g ** 2)

m_hat = self._mb / (1.0 - 0.9**self._tb)

v_hat = self._vb / (1.0 - 0.999**self._tb)

self.b -= lr * m_hat / (np.sqrt(v_hat) + 1e-8)
```

```
class LSTMModel:
```

```
    def __init__(self, input_size, hidden_size, horizon, seed=None):
```

```
        self.hidden_size = hidden_size
```

```
        self.cell = LSTMCell(input_size, hidden_size, seed=seed)
```

```
        self.head = LinearLayer(hidden_size, horizon, seed=seed)
```

```
    def forward(self, inputs):
```

```
        h = np.zeros((self.hidden_size, 1))
```

```
        c = np.zeros((self.hidden_size, 1))
```

```
        caches_seq = []
```

```
        for x in inputs:
```

```
            h, c, cache = self.cell.forward(x, h, c)
```

```

        caches_seq.append(cache)

    y, lin_cache = self.head.forward(h)

    return y, {"seq": caches_seq, "linear": lin_cache}

def backward(self, pred, target, caches):

    err = pred - target

    n = pred.size

    loss = float(np.mean(err ** 2))

    dy = (2.0 / n) * err

    self.cell.zero_grad()

    self.head.zero_grad()

    dh = self.head.backward(caches["linear"], dy)

    dc = np.zeros((self.hidden_size, 1))

    for t in reversed(range(len(caches["seq"]))):

        dh, dc = self.cell.backward(caches["seq"][t], dh, dc)

    return loss

def apply_updates(self, lr):

```

```
self.cell.update(lr)
```

```
self.head.update(lr)
```

```
import argparse
```

```
import os
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
def per_step_rmse(pred, tgt):
```

```
    d = pred - tgt
```

```
    return np.sqrt(np.mean(d**2, axis=0))
```

```
def main():
```

```
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument("exp_id")
```

```
parser.add_argument("--out_dir", default="outputs")

parser.add_argument("--plot_dir", default="plots")

args = parser.parse_args()


os.makedirs(args.plot_dir, exist_ok=True)


pred_path = os.path.join(args.out_dir, f"{args.exp_id}_predictions.npy")

tgt_path = os.path.join(args.out_dir, f"{args.exp_id}_targets.npy")

meta_path = os.path.join(args.out_dir, f"{args.exp_id}_meta.npz")


if not os.path.isfile(pred_path) or not os.path.isfile(tgt_path):

    raise FileNotFoundError(f"Missing {pred_path} or {tgt_path}. Run train.py first.")


pred = np.load(pred_path)

tgt = np.load(tgt_path)

if pred.shape != tgt.shape:

    raise ValueError(f"Shape mismatch pred {pred.shape} vs tgt {tgt.shape}")


horizon = pred.shape[1]
```

```
if os.path.isfile(meta_path):

    meta = np.load(meta_path, allow_pickle=True)

    if int(meta["horizon"]) != horizon:

        raise ValueError("meta horizon does not match prediction width")


steps = np.arange(1, horizon + 1)


fig1, ax1 = plt.subplots(figsize=(10, 4))

ax1.plot(tgt[:, 0], label="Actual (t+1)", color="tab:blue", alpha=0.85)

ax1.plot(pred[:, 0], label="Predicted (t+1)", color="tab:orange", alpha=0.85)

ax1.set_title(f"Test set: actual vs predicted close (t+1) – {args.exp_id}")

ax1.set_xlabel("Test window index")

ax1.set_ylabel("Price")

ax1.legend()

ax1.grid(True, alpha=0.3)

fig1.tight_layout()

p1 = os.path.join(args.plot_dir, f"{args.exp_id}_plot1_t1_series.png")

fig1.savefig(p1, dpi=150)

plt.close(fig1)
```

```
rmse_steps = per_step_rmse(pred, tgt)

fig2, ax2 = plt.subplots(figsize=(8, 4))

ax2.bar(steps, rmse_steps, color="tab:green", alpha=0.85)

ax2.set_title(f"RMSE by forecast step – {args.exp_id}")

ax2.set_xlabel("Step (t+k)")

ax2.set_ylabel("RMSE (same units as price)")

ax2.set_xticks(steps)

ax2.grid(True, axis="y", alpha=0.3)

fig2.tight_layout()

p2 = os.path.join(args.plot_dir, f"{args.exp_id}_plot2_rmse_by_step.png")

fig2.savefig(p2, dpi=150)

plt.close(fig2)


sample_idx = pred.shape[0] // 2

fig3, ax3 = plt.subplots(figsize=(8, 4))

ax3.plot(steps, tgt[sample_idx], marker="o", label="Actual", color="tab:blue")

ax3.plot(steps, pred[sample_idx], marker="o", label="Predicted", color="tab:orange")

ax3.set_title(f"Single window (index {sample_idx}): multi-step forecast – {args.exp_id}")
```

```
ax3.set_xlabel("Horizon step")

ax3.set_ylabel("Price")

ax3.set_xticks(steps)

ax3.legend()

ax3.grid(True, alpha=0.3)

fig3.tight_layout()

p3 = os.path.join(args.plot_dir, f"{args.exp_id}_plot3_single_window.png")

fig3.savefig(p3, dpi=150)

plt.close(fig3)
```

```
print(f"Saved plots:\n {p1}\n {p2}\n {p3}")
```

```
if __name__ == "__main__":
```

```
    main()
```

```
import argparse
```

```
import csv
```

```
import os
```

```
from datetime import datetime, timezone
```

```
import numpy as np
```

```
import yfinance as yf
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from lstm import LSTMModel
```

```
TICKER = "AAPL"
```

```
HIDDEN = 64
```

```
SEQ_LEN = 30
```

```
HORIZON = 5
```

```
EPOCHS = 100
```

```
LR = 1e-3
```

```
SPLIT = 0.8
```

```
SEED = 42
```

```
START = "2015-01-01"
```

```
END = "2025-01-01"
```

```
OUT_DIR = "outputs"
```

```
def fetch_close_prices(ticker, start, end):

    df = yf.download(ticker, start=start, end=end, progress=False, auto_adjust=True)

    if df.empty:

        raise RuntimeError(f"No data returned for {ticker} ({start} to {end}).")

    return df["Close"].to_numpy(dtype=np.float64).reshape(-1, 1)
```

```
def build_windows(scaled, seq_len, horizon, start_idx, end_idx):

    inputs_batch = []

    targets_batch = []

    last_i = end_idx - seq_len - horizon

    for i in range(start_idx, last_i + 1):

        seq_in = [scaled[i + t : i + t + 1].copy() for t in range(seq_len)]

        tgt = scaled[i + seq_len : i + seq_len + horizon].copy()

        inputs_batch.append(seq_in)

        targets_batch.append(tgt)

    return inputs_batch, targets_batch
```

```
def inverse_matrix(scaler, arr):  
  
    flat = arr.reshape(-1, 1)  
  
    return scaler.inverse_transform(flat).reshape(arr.shape)
```

```
def rmse_mape(pred, actual):  
  
    diff = pred - actual  
  
    rmse = float(np.sqrt(np.mean(diff**2)))  
  
    denom = np.maximum(actual, 1e-8)  
  
    mape = float(np.mean(np.abs(diff / denom)) * 100.0)  
  
    return rmse, mape
```

```
def evaluate(model, inputs_list, targets_list, scaler):  
  
    preds = []  
  
    tgts = []  
  
    for x_seq, y in zip(inputs_list, targets_list):  
  
        y_hat, _ = model.forward(x_seq)
```

```
preds.append(y_hat.copy())
```

```
tgts.append(y.copy())
```

```
pred_scaled = np.stack([p.reshape(-1) for p in preds], axis=0)
```

```
tgt_scaled = np.stack([t.reshape(-1) for t in tgts], axis=0)
```

```
pred_inv = inverse_matrix(scaler, pred_scaled)
```

```
tgt_inv = inverse_matrix(scaler, tgt_scaled)
```

```
return (*rmse_mape(pred_inv, tgt_inv), pred_inv, tgt_inv)
```

```
def main():
```

```
    ap = argparse.ArgumentParser()
```

```
    ap.add_argument("--exp_id", default="default")
```

```
    ap.add_argument("--log", default="experiment_log.csv")
```

```
    ap.add_argument("--ticker", default=None, help="override TICKER in this file")
```

```
    args = ap.parse_args()
```

```
    ticker = args.ticker or TICKER
```

```
    os.makedirs(OUT_DIR, exist_ok=True)
```

```
close = fetch_close_prices(ticker, START, END)

n = len(close)

scaler = MinMaxScaler()

scaled = scaler.fit_transform(close)

train_end = int(n * SPLIT)

train_inputs, train_targets = build_windows(scaled, SEQ_LEN, HORIZON, 0, train_end)

test_inputs, test_targets = build_windows(scaled, SEQ_LEN, HORIZON, train_end, n)

if not train_inputs:

    raise RuntimeError("No training windows.")

if not test_inputs:

    raise RuntimeError("No test windows.")

model = LSTMModel(1, HIDDEN, HORIZON, seed=SEED)

train_indices = np.arange(len(train_inputs))

best_rmse = float("inf")
```

```
best_epoch = -1
```

```
last_train_loss = 0.0
```

```
last_test_rmse = 0.0
```

```
last_test_mape = 0.0
```

```
for epoch in range(1, EPOCHS + 1):
```

```
    np.random.default_rng(SEED + epoch).shuffle(train_indices)
```

```
    losses = []
```

```
    for idx in train_indices:
```

```
        x_seq = train_inputs[idx]
```

```
        y = train_targets[idx]
```

```
        pred, caches = model.forward(x_seq)
```

```
        losses.append(model.backward(pred, y, caches))
```

```
        model.apply_updates(LR)
```

```
last_train_loss = float(np.mean(losses))
```

```
test_rmse, test_mape, pred_inv, tgt_inv = evaluate(model, test_inputs, test_targets, scaler)
```

```
last_test_rmse = test_rmse
```

```
last_test_mape = test_mape
```

```
if test_rmse < best_rmse:

    best_rmse = test_rmse

    best_epoch = epoch


last_pred_inv = pred_inv.copy()

last_tgt_inv = tgt_inv.copy()


if epoch == 1 or epoch % 10 == 0 or epoch == EPOCHS:

    print(

        f"epoch {epoch}/{EPOCHS}  train_loss={last_train_loss:.6f}  "

        f"test_rmse={test_rmse:.6f}  test_mape={test_mape:.4f}%"

    )
```

```
pred_path = os.path.join(OUT_DIR, f"{args.exp_id}_predictions.npy")
```

```
tgt_path = os.path.join(OUT_DIR, f"{args.exp_id}_targets.npy")
```

```
np.save(pred_path, last_pred_inv)
```

```
np.save(tgt_path, last_tgt_inv)
```

```
meta_path = os.path.join(OUT_DIR, f"{args.exp_id}_meta.npz")
```

```
np.savez(  
  
    meta_path,  
  
    horizon=HORIZON,  
  
    seq_len=SEQ_LEN,  
  
    best_epoch=best_epoch,  
  
    best_test_rmse=best_rmse,  
  
)
```

```
row = {  
  
    "timestamp": datetime.now(timezone.utc).isoformat(),  
  
    "exp_id": args.exp_id,  
  
    "ticker": ticker,  
  
    "hidden": HIDDEN,  
  
    "seq_len": SEQ_LEN,  
  
    "horizon": HORIZON,  
  
    "epochs": EPOCHS,  
  
    "lr": LR,  
  
    "split": SPLIT,  
  
    "seed": SEED,
```

```
    "train_loss_final": last_train_loss,  
  
    "test_rmse_final": last_test_rmse,  
  
    "test_mape_final": last_test_mape,  
  
    "test_rmse_best": best_rmse,  
  
    "best_epoch": best_epoch,  
  
    "pred_path": pred_path,  
  
    "tgt_path": tgt_path,  
  
}
```

```
log_path = args.log
```

```
new_file = not os.path.isfile(log_path)
```

```
with open(log_path, "a", newline="") as f:
```

```
    w = csv.DictWriter(f, fieldnames=list(row.keys()))
```

```
    if new_file:
```

```
        w.writeheader()
```

```
    w.writerow(row)
```

```
print(f"Saved predictions to {pred_path} and log to {log_path}")
```

```
if __name__ == "__main__":
```

```
    main()
```